Test-to-property matrix (MS3)

We use this table to connect the specification to the tests a reviewer can run and extend. `make test` runs the suite through Docker; the host also needs `make` and a POSIX shell. The final run's logs are in results/ms3-logs/. Scheduler events, worker and proxy logs, and snapshots for that run are under `results/ms3-logs/evidence-make-test-5/`, and the course-cluster runs are under `results/ms3-logs/hokea-cluster/`. A test checks its recorded scenario; the table does not claim exhaustive coverage of every possible execution.

Property codes

Code	Property (spec section)
S1	No task is marked complete unless the worker currently holding its unexpired lease reported success, with every declared output published and verified (§3).
S2	No task is assigned before all its parents succeeded, and no worker starts executing before the scheduler acknowledged its start (§3).
S3	At most one accepted success per logical task and at most one owner at a time; retries, replays and late messages never add a second completion (§3).
L1	While a worker is available, every ready task is eventually assigned and then succeeds or is retried; a slow worker that keeps renewing is not interrupted (§4).
L2	A task whose worker crashes, freezes or is cut off becomes ready again once its lease expires and is reassigned (§4).
P-REC	Recovery bound: never detected before `LEASE_TIMEOUT_MS`; reassigned within `LEASE_TIMEOUT_MS` + 2 s when an idle worker is polling (§6).
C	Interface contract: validation, limits, strict JSON, idempotent receipts, errors (§1, api.md).
E	Evidence integrity: the history checker itself, export completeness (§3).

Fault codes: **crash** (SIGKILL), **restart** (new process/session), **pause** (SIGSTOP, longer than the lease), **loss** (dropped request or reply), **delay**, **dup** (delivered twice), **part** (link partition), **run** (scheduler restart), **store** (artifact store down).

Fault-injection suites (`tests/faults/`, `make faults`)

Test	Fault injected	Properties
`test_worker_crash.py::test_worker_crash_reassignment` (MS2's oracle, now passing; also on the course cluster)	crash during the operation	L2, P-REC, S1, S2, S3
`test_crash_rerun_bounded.py::test_dead_worker_task_rerun_within_budget` (requested in the MS2 feedback; also on the course cluster)	crash during an ordinary 20 s task, no test hooks, a second worker already polling; the task must run again within 60 s and succeed once	L2, S1, S3
`test_lease_faults.py::test_crash_at_each_attempt_stage_is_recovered[before_start]`	crash after claim, before start	L2, P-REC, S1, S3
`…[operation]`	crash while executing	L2, P-REC, S1, S3
`…[before_report]`	crash after uploading outputs, before reporting (orphan output must not count)	L2, P-REC, S1, S3
`test_lease_faults.py::test_restarted_worker_is_a_new_session_and_the_old_identity_stays_dead`	crash + restart; late start/heartbeat/report with the dead identity	L2, S1, S3
`test_lease_faults.py::test_paused_worker_loses_its_lease_and_its_late_work_is_rejected`	pause longer than the lease, then resume	L2, S1, S3, L1 (worker serves again)
`test_lease_faults.py::test_slow_worker_with_heartbeats_keeps_its_task`	none: task runs 2.4 leases	L1 (slow is not dead), S3

Test	Fault injected	Properties
`test_network_faults.py::test_lost_heartbeats_expire_the_lease_and_the_late_success_is_rejected`	loss of every heartbeat; the worker really finishes and reports late	S1, S3, L2
`test_network_faults.py::test_partition_longer_than_the_lease_moves_the_task`	part (longer than lease), heal	L2, S3, L1 (worker serves again)
`test_network_faults.py::test_partition_shorter_than_the_lease_keeps_the_attempt`	part (2 s)	L1 (no false expiry), S3
`test_network_faults.py::test_lost_claim_and_start_replies_are_replayed_from_receipts`	loss of a claim reply and a start reply	S3, C (receipts), L1
`test_network_faults.py::test_crash_after_success_is_accepted_but_its_reply_is_lost`	loss of report reply + crash (report boundary)	S3, L1
`test_network_faults.py::test_delayed_and_duplicated_messages_change_nothing`	delay (0–150 ms) on all messages, dup on every start and report	S3, S1, L1
`test_network_faults.py::test_chaos_campaign_keeps_every_guarantee`	seeded mix: 5% request loss, 5% reply loss, delay, a crash of a worker held in a running attempt, a pause longer than the lease of a worker holding a task, a replacement worker; both victims' attempts must expire	S1, S2, S3, L1, L2

We also run `check_history` on each fault test's scheduler and worker history. That checks S1–S3 in those tests even where the row highlights a different property.

Integration suites (`tests/integration/`)

Test	Fault	Properties
`test_functional_workload.py::test_committed_functional_workload`	none	L1, S1–S3 (`result=22`)
`test_system.py::test_video_pipeline`	none	L1, S1–S3 (real FFmpeg outputs)
`test_system.py::test_functional_multiple_jobs_workers`	none	L1, S1–S3
`test_system.py::test_join_and_independent_branch_concurrency`	none	S2, L1 (parallel branches)
`test_system.py::test_multiple_roots_sinks_and_all_tasks_required`	none	S2, job completion semantics
`test_system.py::test_simultaneous_claims_and_session_single_slot`	concurrent claims	S3 (one owner)
`test_system.py::test_submission_and_claim_receipts`	dup (replayed submission and claim)	C, S3
`test_system.py::test_cached_empty_claim_stays_empty`	dup	C
`test_system.py::test_real_worker_failure_attempt_two`	explicit operation failure	L1 (retry), S3
`test_system.py::test_duplicate_conflicting_stale_reports_and_retry`	dup, stale and conflicting reports	S3, S1
`test_system.py::test_success_requires_published_correct_outputs`	missing or wrong output	S1
`test_system.py::test_artifact_staging_atomic_publication_and_conflicts`	partial upload, conflicting bytes	S1 (immutable outputs)

Test	Fault	Properties
`test_system.py::test_invalid_dags_never_accepted[9 cases]`	invalid input	C
`test_system.py::test_strict_json_types_and_unknown_fields`	invalid input	C
`test_worker_client.py::test_java_client_upload_submit_status_fetch`	none	L1, C (client)
`test_worker_client.py::test_worker_has_one_slot_and_starts_only_after_ack`	none	S2, S3
`test_worker_client.py::test_hooks_are_disabled_by_default`	none	C (test hooks off in production)
`test_worker_client.py::test_local_operation_timeout_is_an_explicit_bounded_failure`	operation timeout	L1 (retry), S3
`test_worker_client.py::test_stale_report_rejection_does_not_kill_the_worker`	attempt revoked while running	S3, L1
`test_worker_client.py::test_lost_report_ack_is_replayed_not_reexecuted`	loss of a report reply	S3
`test_worker_client.py::test_worker_abandons_old_run_and_joins_the_new_one`	run	S1, S3 (stale run rejected), L1
`test_api.py::test_stale_run_is_rejected_everywhere`	run	S1, S3
`test_api.py::test_storage_unavailable_is_transient_and_accepts_nothing`	store	S1, C
`test_api.py::test_rejected_commands_do_not_change_snapshots`	invalid commands	S1, S3
`test_api.py::test_source_descriptors_are_verified_before_acceptance`	missing/mismatched input	C, S1
`test_api.py` (health, structured errors, 413 limit, counters/snapshots)	none	C, E
`test_hokea_smoke.py::test_service_exchange`	none	C (deployment adapter)

Unit suites (no services)

Test file	What it checks	Properties
`tests/unit/test_history_checks.py` (16 tests)	Each history invariant is falsified by a one-event mutation of a real saved history, including MS3's "success from an expired attempt" and "expiry of a non-owner".	E (the checker can detect S1–S3 violations)
`tests/unit/test_fault_oracle.py`	The crash oracle's preconditions and polling evidence cannot be satisfied by malformed or stale evidence.	E
`tests/unit/test_hokea_adapter.py`, `test_hokea_packaging.py`	Course-cluster adapter and runner packaging; a runner whose Hokea differs from the public commit is recorded, not rejected; a Hokea missing a call the adapter makes fails before any pod starts (the two tests against a real Hokea skip where it is not installed).	C

JUnit (`mvn -B verify`, also part of `make test`)

Class	Tests	Properties
`LeaseTest` (MS3, injected clock, no sleeps)	16: reassignment after expiry; expiry exactly at the deadline; heartbeats keep a slow owner; start renews; crash before start; every late message from an expired attempt is rejected with no state change; late success rejected even before anyone else claims; expiry between artifact verification and commit; accepted results never expire; resumed session gets only a new attempt; claim replay after expiry; non-owner heartbeats; FIFO order of expired tasks; recovery latency measurement; explicit failure still retries immediately; an 8-thread crash/expiry/report race	S1, S3, L1, L2, P-REC
`SchedulerStateMachineTest`	12: idle polling leaves no history (memory stays flat), atomic rejection, retry numbering without cap, edge accounting, deterministic FIFO, job completion, session slot, replay, report shape, 8-thread stress	S1, S2, S3, L1, C
`SchedulerCoreTest`	8: transitions, wrong owner/stale run, duplicate failures, FIFO tail, atomic claims, revalidation after a concurrent failure, claim receipts, gap-free events	S1, S2, S3, C, E
`WireContractTest`, `ManifestValidatorTest`	33: DAG validation, limits, strict JSON, semantic equality, identities, key namespaces	C

Class	Tests	Properties
`ArtifactStoreTest`	8: immutable publication, integrity checks, partial uploads invisible, concurrent publication	S1 (outputs cannot change after acceptance), C

How to extend

To add a service-backed test, use the `system` fixture. It starts an isolated scheduler and artifact store and exports evidence when the test exits. For example:

```
from tests.harness.api import check_history, manifest, task

def test_my_fault(system):
    a = system.start_worker(proxy=True)            # proxy=True: this worker's scheduler link is faultable
    system.proxy(a).add("drop_request", path="/v1/attempts/heartbeat")
    m = manifest([task("X", delay_ms=3000)])
    done = system.api.complete(system.api.submit(m), timeout=60)
    check_history(m, done, system.api.events(), system.worker_events(a))
```

We provide these fault hooks: `kill_worker`, `pause_worker` / `resume_worker`, `restart_scheduler`, `stop_artifacts`, worker stall points (`ENABLE_TEST_HOOKS=true`, `TEST_GATE_TASK`, `TEST_GATE_STAGE` = `before_start` | `operation` | `before_report`), and per-link proxy rules (`drop_request`, `drop_response`, `delay`, `duplicate`, `partition()`/`heal()`). The event schema and offline-check instructions are in evidence.md.